Support Agent — Build Plan

Chat-only multi-agent support platform · LangGraph · pgvector · OpenAI · written 2026-10-11

- Overview
  - Rules of the game
  - Phases
  - Architecture
  - Life of one chat message
  - Decisions (and why)
- Phase 0 — Setup
  - 0.1 Python + project
  - 0.2 Files
  - Verify
- Phase 1 — Database
  - 1.1 Engine and session
  - 1.2 Models
  - 1.3 Create-tables script
  - Verify
- Phase 2 — Auth + admin API
  - 2.1 Security helpers
  - 2.2 API schemas
  - 2.3 Routers
  - 2.4 App entry point and admin seed
  - Verify
- Phase 3 — The DAG spec
  - 3.1 What a DAG file looks like
  - 3.2 Schema + validator
  - 3.3 Versioned DAG API
  - 3.4 Tests
  - Verify
- Phase 4 — Knowledge base (pgvector)
  - 4.1 OpenAI clients
  - 4.2 Chunk, embed, search
  - 4.3 Knowledge API
  - Verify
- Phase 5 — Graph compiler (the heart)
  - 5.1 State
  - 5.2 Tools (replace the phase 3 stub)
  - 5.3 Node builders
  - 5.4 Compiler
  - 5.5 Registry (cache + hot reload + memory)
  - 5.6 Wire it into the app
  - 5.7 Terminal chat
  - Verify
- Phase 6 — Chat API with streaming

6.1 Chat router
6.2 Register the router
Verify
Phase 7 — Frontend: admin panel + chat
7.1 Serve the pages
7.2 Shared helper
7.3 Admin panel
7.4 Chat page
Verify
Phase 8 — After it works
8.1 See inside the graph: LangSmith tracing (15 minutes)
8.2 Schema migrations: Alembic
8.3 Long conversations
8.4 New node types
8.5 Per-client knowledge upload
8.6 Evals
8.7 Ship it

Overview

A chat-only, multi-agent customer-support platform. One admin panel onboards people and clients. Each client gets its own **DAG file** (a JSON pipeline spec) that is compiled at runtime into a **LangGraph** graph. Postgres with **pgvector** stores everything: clients, users, DAG versions, knowledge-base chunks (embeddings), and LangGraph's own conversation memory. One OpenAI key in `.env` serves every client.

You type all application code yourself. These docs give the code and explain it. I never write files under `app/`, `scripts/`, `static/`, or `examples/`.

Rules of the game

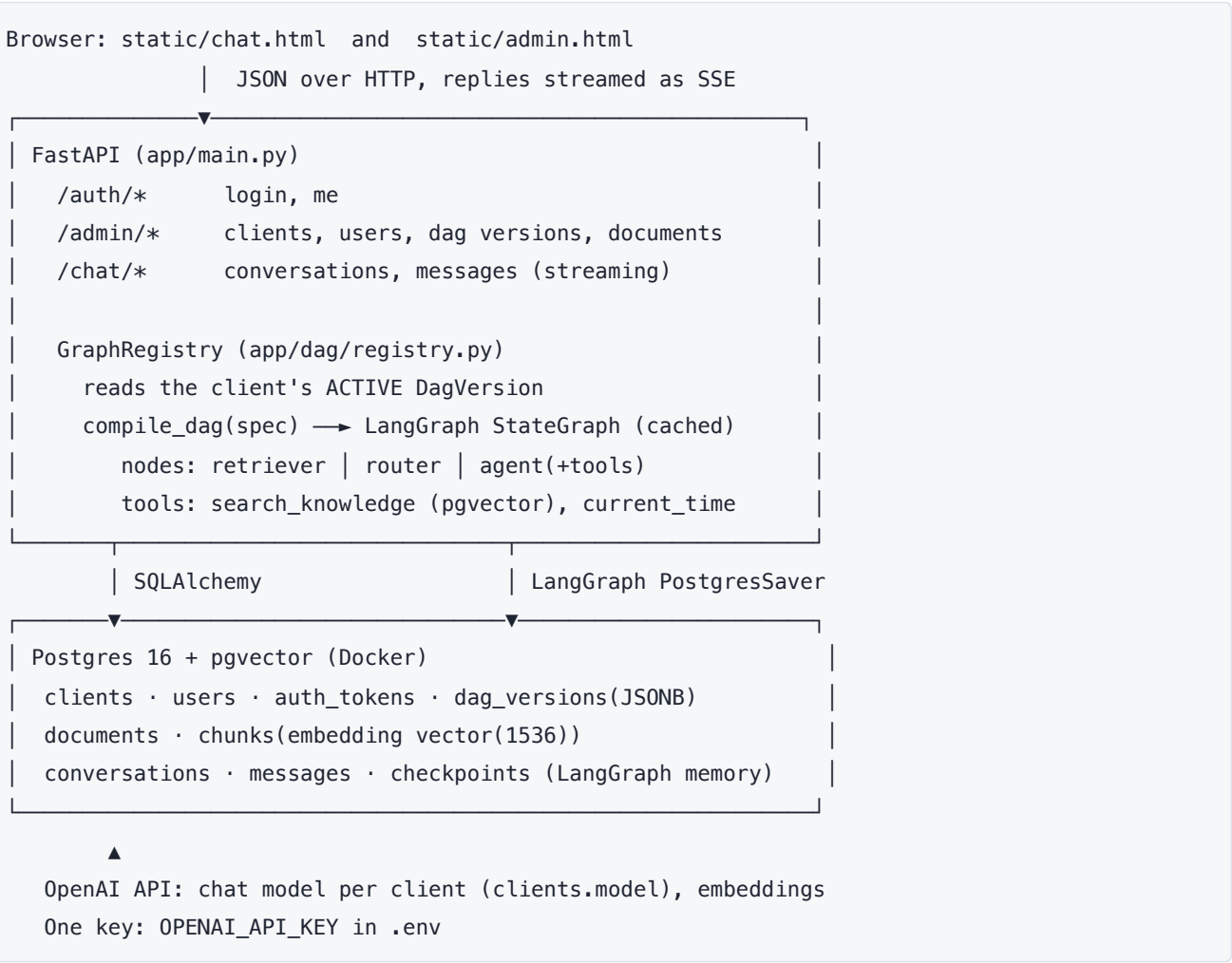
1. One phase per sitting. Each phase ends with a **Verify** block that proves it works.
2. Commit after every green Verify: `git add -A && git commit -m "phase N: ..."`.
3. If a Verify fails, paste the exact error to me. Do not skip ahead.
4. Type the code; read the explanation *after* typing, then re-read the code.
5. Every phase lists **What you learn** so you can check your own understanding.

Phases

#	Phase	You end up with
0	Setup	uv project, Python 3.12, Postgres+pgvector in Docker, settings
1	Database	SQLAlchemy models, tables created, vector index
2	Auth + admin API	login, tokens, create clients and users via curl
3	DAG spec	Pydantic DAG schema with cycle detection, versioned DAG API, tests
4	Knowledge base	documents → chunks → embeddings → similarity search
5	Graph compiler	spec → LangGraph, agents with tools, hot reload, terminal chat
6	Chat API	conversations + streaming SSE endpoint
7	Frontend	admin panel + chat page, vanilla HTML/JS
8	What's next	tracing, migrations, handoff, evals

Target: phases 0–5 give you a working multi-agent chat in the terminal. 6–7 put it on the web.

Architecture



Life of one chat message

1. `POST /chat/conversations/{id}/messages` arrives with a Bearer token.
2. The token resolves to a user → their client → the client's active `DagVersion`.
3. `GraphRegistry.get()` returns the compiled graph for that version (compiles on first use or after a DAG change).
4. The graph runs with `thread_id = conversation id`. LangGraph's Postgres checkpointer loads the previous messages for that thread automatically.
5. Nodes run in DAG order: `retrieve` (pgvector) → `triage` (router, structured output) → one `agent` (LLM, may call tools) → END.
6. Tokens and node events stream back as SSE. The final answer is also saved to `messages` for the admin's view.

Decisions (and why)

Decision	Why
DAG spec is JSON stored in a JSONB column, versioned	Editable from the admin panel, diffable, rollback by activating an older version. No file system needed.
Strict DAG at the top level (no cycles)	You asked for a DAG. Cycles live only *inside* an agent node (its tool loop), which has a hard round limit.
Agent tool loop written by hand (~20 lines)	You see the real ReAct loop instead of a black box, and the API won't shift under you.
Router uses structured output with a `Literal` of labels	The model can only answer with a valid label, so the conditional edge can't fall off the map.
Sync SQLAlchemy + sync FastAPI endpoints	Less to learn, no event-loop foot-guns. FastAPI runs `def` endpoints in a threadpool.
LangGraph `PostgresSaver` for memory	Conversation history is handled by the framework, keyed by `thread_id`. Zero custom code.
One `OPENAI_API_KEY`, model chosen per client	Exactly what you asked for. Per-node model override is in the spec for later.
Vanilla HTML/JS frontend	No node toolchain in the critical path. Replace with React later if you want.

Phase 0 — Setup

Goal: a uv-managed Python 3.12 project, Postgres + pgvector running in Docker, and a typed settings object that loads `.env`.

What you learn: uv projects, why Python 3.12 (wheels for every dependency exist), docker compose, pydantic-settings.

0.1 Python + project

```
cd ~/Desktop/Support-Agent
uv python install 3.12
uv init --python 3.12 --name support_agent
rm -f main.py
```

`uv init` created `pyproject.toml` and `.python-version`. Now add dependencies:

```
uv add fastapi "uvicorn[standard]" sqlalchemy "psycopg[binary,pool]" pgvector pydantic-settings
bcrypt langgraph langchain-core langchain-openai langgraph-checkpoint-postgres
uv add --dev pytest httpx
```

What each one is for:

Package	Role
fastapi, uvicorn	HTTP API and the server that runs it
sqlalchemy, psycopg	ORM and the Postgres driver (psycopg 3). `pool` extra gives the connection pool LangGraph's checkpointer uses
pgvector	SQLAlchemy `Vector` column type + distance operators
pydantic-settings	Load `.env` into a typed `Settings` object
bcrypt	Password hashing
langgraph, langchain-core	Graph runtime, message types, tool decorator
langchain-openai	`ChatOpenAI`, `OpenAIEmbeddings`
langgraph-checkpoint-postgres	Conversation memory stored in Postgres
pytest, httpx	Tests (httpx is FastAPI's test client transport)

0.2 Files

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
.pytest_cache/
```

`.env.example` (commit this; copy it to `.env` and fill the key)

```
OPENAI_API_KEY=sk-...
DATABASE_URL=postgresql+psycopg://support:support@127.0.0.1:5433/support
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=change-me-now
DEFAULT_MODEL=gpt-4.1-mini
EMBEDDING_MODEL=text-embedding-3-small
```

```
cp .env.example .env
# then edit .env and paste your real OpenAI key
```

`docker-compose.yml`

```
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: support
      POSTGRES_PASSWORD: support
      POSTGRES_DB: support
    ports:
      - "127.0.0.1:5433:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Port 5433 on the host avoids colliding with any local Postgres on 5432. The image is plain Postgres 16 with the `vector` extension compiled in; you still have to run `CREATE EXTENSION vector` once (phase 1 does it).

`app/__init__.py` — empty file.

`app/config.py`

```
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
```

```
    openai_api_key: str
    database_url: str
    admin_email: str
    admin_password: str
    default_model: str = "gpt-4.1-mini"
    embedding_model: str = "text-embedding-3-small"
    embedding_dims: int = 1536

    @property
    def psycopg_url(self) -> str:
        """Plain libpq URL for tools that don't speak SQLAlchemy (the LangGraph
checkpointer)."""
        return self.database_url.replace("postgresql+psycopg://", "postgresql://")


settings = Settings()
```

Explanation. `BaseSettings` reads each field from the environment (case-insensitive), falling back to `.env`. It does *not* export the values into `os.environ`, which is why later we pass `settings.openai_api_key` to `ChatOpenAI` explicitly instead of hoping the library finds the env var. `embedding_dims` must match the embedding model: `text-embedding-3-small` produces 1536 floats, and the vector column is declared with that size.

Verify

```
open -a Docker
```

Wait until the whale icon is steady, then:

```
docker compose up -d && docker compose ps
```

```
uv run python -c "from app.config import settings; print(settings.default_model,
settings.psycopg_url)"
```

Expected: the container shows `running`, and the second command prints `gpt-4.1-mini postgresql://support:support@127.0.0.1:5433/support`.

Commit: `git add -A && git commit -m "phase 0: project setup"`

Phase 1 — Database

Goal: every table the system needs, created by one script, including the `vector` extension and an HNSW index on embeddings.

What you learn: SQLAlchemy 2.0 declarative models, UUID keys, JSONB, the pgvector column type, why `expire_on_commit=False`, approximate-nearest-neighbour indexes.

1.1 Engine and session

`app/db.py`

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.config import settings

engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, expire_on_commit=False)


class Base(DeclarativeBase):
    pass


def get_db():
    """FastAPI dependency: one session per request, always closed."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Explanation. `pool_pre_ping` checks a pooled connection is alive before using it, so a restarted Docker DB doesn't produce one mysterious failure. `expire_on_commit=False` means objects keep their loaded values after `commit()`; without it, touching `conv.id` after the session closes (which we do inside a streaming response) raises.

1.2 Models

`app/models.py`

```
import uuid
from datetime import datetime, timezone

from pgvector.sqlalchemy import Vector
```

```
from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String, Text, UniqueConstraint
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.config import settings
from app.db import Base


def utcnow() -> datetime:
    return datetime.now(timezone.utc)


def uuid_pk():
    return mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)


def created_at_col():
    return mapped_column(DateTime(timezone=True), default=utcnow)


class Client(Base):
    __tablename__ = "clients"

    id: Mapped[uuid.UUID] = uuid_pk()
    slug: Mapped[str] = mapped_column(String(64), unique=True)   # the "client ID" people type
    name: Mapped[str] = mapped_column(String(200))
    model: Mapped[str] = mapped_column(String(100))              # OpenAI model for this client
    created_at: Mapped[datetime] = created_at_col()


class User(Base):
    __tablename__ = "users"

    id: Mapped[uuid.UUID] = uuid_pk()
    email: Mapped[str] = mapped_column(String(320), unique=True)
    password_hash: Mapped[str] = mapped_column(String(200))
    role: Mapped[str] = mapped_column(String(20))                # "admin" | "member"
    client_id: Mapped[uuid.UUID | None] = mapped_column(ForeignKey("clients.id"), nullable=True)
    created_at: Mapped[datetime] = created_at_col()


class AuthToken(Base):
    __tablename__ = "auth_tokens"

    token: Mapped[str] = mapped_column(String(64), primary_key=True)
    user_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    created_at: Mapped[datetime] = created_at_col()
```

```
    user: Mapped[User] = relationship()


class DagVersion(Base):
    __tablename__ = "dag_versions"
    __table_args__ = (UniqueConstraint("client_id", "version"),)

    id: Mapped[uuid.UUID] = uuid_pk()
    client_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("clients.id", ondelete="CASCADE"))
    version: Mapped[int] = mapped_column(Integer)
    spec: Mapped[dict] = mapped_column(JSONB)                    # the DAG file itself
    is_active: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[datetime] = created_at_col()


class Document(Base):
    __tablename__ = "documents"

    id: Mapped[uuid.UUID] = uuid_pk()
    client_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("clients.id", ondelete="CASCADE"))
    title: Mapped[str] = mapped_column(String(200))
    created_at: Mapped[datetime] = created_at_col()

    chunks: Mapped[list["Chunk"]] = relationship(cascade="all, delete-orphan")


class Chunk(Base):
    __tablename__ = "chunks"

    id: Mapped[uuid.UUID] = uuid_pk()
    document_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("documents.id",
ondelete="CASCADE"))
    client_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("clients.id", ondelete="CASCADE"))
    position: Mapped[int] = mapped_column(Integer)
    content: Mapped[str] = mapped_column(Text)
    embedding: Mapped[list[float]] = mapped_column(Vector(settings.embedding_dims))


class Conversation(Base):
    __tablename__ = "conversations"

    id: Mapped[uuid.UUID] = uuid_pk()
    client_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("clients.id", ondelete="CASCADE"))
    user_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    title: Mapped[str] = mapped_column(String(200))
    created_at: Mapped[datetime] = created_at_col()
```

```
class Message(Base):
    __tablename__ = "messages"

    id: Mapped[uuid.UUID] = uuid_pk()
    conversation_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("conversations.id",
ondelete="CASCADE"))
    role: Mapped[str] = mapped_column(String(20))                # "user" | "assistant"
    content: Mapped[str] = mapped_column(Text)
    created_at: Mapped[datetime] = created_at_col()
```

Explanation.

- `Client.slug` is the human "client ID" (e.g. `acme`); `Client.id` is the UUID used in foreign keys. `Client.model` is the OpenAI model name you pick in the admin panel.
- `DagVersion` holds the DAG file as JSONB. Every save creates a new row with `version + 1`; exactly one row per client has `is_active=True`. That gives you history and rollback for free.
- `Chunk.client_id` is duplicated from the document on purpose: similarity search filters by client, and a join on every query would be slower and easier to get wrong (tenant isolation).
- `Chunk.embedding` is a `Vector(1536)`. pgvector stores it as a native type and gives you `<=>` (cosine distance) in SQL.
- `Message` is a plain log for humans (admin view, conversation list). LangGraph keeps its own richer state (tool calls etc.) in its checkpoint tables.

1.3 Create-tables script

`scripts/__init__.py` — empty file (so `python -m scripts.init_db` resolves imports from the repo root).

`scripts/init_db.py`

```
from sqlalchemy import text

from app import models  # noqa: F401  (importing registers the tables on Base.metadata)
from app.db import Base, engine

with engine.begin() as conn:
    conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector"))

Base.metadata.create_all(engine)

with engine.begin() as conn:
    conn.execute(text(
        "CREATE INDEX IF NOT EXISTS chunks_embedding_idx "
        "ON chunks USING hnsw (embedding vector_cosine_ops)"
    ))

print("tables:", ", ".join(sorted(Base.metadata.tables)))
```

Explanation. `create_all` is idempotent: it creates missing tables and leaves existing ones alone (it does *not* alter columns; when you change a model later, phase 8 introduces Alembic migrations). The HNSW index makes nearest-neighbour search approximate but fast; with a few thousand chunks you wouldn't notice the difference, but it's the right habit.

Verify

```
uv run python -m scripts.init_db
```

```
docker compose exec db psql -U support -d support -c '\dt' -c '\d chunks'
```

Expected: eight tables listed (`auth_tokens, chunks, clients, conversations, dag_versions, documents, messages, users`), and `chunks` shows `embedding | vector(1536)` and the `chunks_embedding_idx` index.

Commit: `git add -A && git commit -m "phase 1: database models"`

Phase 2 — Auth + admin API

Goal: an admin can log in, create clients (name, slug, model) and onboard users (members bound to a client). All through HTTP, tested with curl.

What you learn: password hashing, opaque bearer tokens, FastAPI dependencies (`Depends`) as the auth mechanism, router-level guards, Pydantic request/response schemas.

2.1 Security helpers

`app/security.py`

```
import secrets

import bcrypt
from fastapi import Depends, Header, HTTPException
from sqlalchemy.orm import Session

from app.db import get_db
from app.models import AuthToken, User


def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()


def verify_password(password: str, password_hash: str) -> bool:
    return bcrypt.checkpw(password.encode(), password_hash.encode())


def create_token(db: Session, user: User) -> str:
    token = AuthToken(token=secrets.token_urlsafe(32), user_id=user.id)
    db.add(token)
    db.commit()
    return token.token


def current_user(
    authorization: str | None = Header(default=None),
    db: Session = Depends(get_db),
) -> User:
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "Missing bearer token")
    token = db.get(AuthToken, authorization.removeprefix("Bearer "))
    if token is None:
        raise HTTPException(401, "Invalid token")
    return token.user
```

```
def require_admin(user: User = Depends(current_user)) -> User:
    if user.role != "admin":
        raise HTTPException(403, "Admin only")
    return user
```

Explanation. A token is just 32 random bytes stored in a table; logging out is deleting the row. No JWT, no clock skew, no secret rotation. `current_user` is a dependency: any endpoint that declares `user: User = Depends(current_user)` gets the user or a 401 before its own code runs. `require_admin` stacks on top of it.

2.2 API schemas

`app/schemas.py`

```
import uuid
from datetime import datetime
from typing import Any, Literal

from pydantic import BaseModel, ConfigDict, Field


class FromORM(BaseModel):
    model_config = ConfigDict(from_attributes=True)


# auth
class LoginIn(BaseModel):
    email: str
    password: str


class TokenOut(BaseModel):
    token: str
    role: str
    client_id: uuid.UUID | None


# users
class UserCreate(BaseModel):
    email: str
    password: str = Field(min_length=8)
    role: Literal["admin", "member"] = "member"
    client_id: uuid.UUID | None = None


class UserOut(FromORM):
    id: uuid.UUID
```

```
    email: str
    role: str
    client_id: uuid.UUID | None


# clients
class ClientCreate(BaseModel):
    slug: str = Field(pattern=r"^[a-z0-9][a-z0-9-]{1,62}$")
    name: str
    model: str | None = None


class ClientUpdate(BaseModel):
    name: str | None = None
    model: str | None = None


class ClientOut(FromORM):
    id: uuid.UUID
    slug: str
    name: str
    model: str
    created_at: datetime


# dag versions (used in phase 3)
class DagIn(BaseModel):
    spec: dict[str, Any]
    activate: bool = True


class DagOut(FromORM):
    id: uuid.UUID
    version: int
    is_active: bool
    created_at: datetime
    spec: dict[str, Any]


# knowledge (phase 4)
class DocumentIn(BaseModel):
    title: str
    text: str = Field(min_length=1)


class DocumentOut(FromORM):
    id: uuid.UUID
    title: str
```

```
    created_at: datetime


class SearchHit(BaseModel):
    document_id: uuid.UUID
    position: int
    content: str


# chat (phase 6)
class ConversationCreate(BaseModel):
    client_id: uuid.UUID | None = None


class ConversationOut(FromORM):
    id: uuid.UUID
    client_id: uuid.UUID
    title: str
    created_at: datetime


class MessageIn(BaseModel):
    content: str = Field(min_length=1)


class MessageOut(FromORM):
    id: uuid.UUID
    role: str
    content: str
    created_at: datetime
```

Explanation. `from_attributes=True` lets FastAPI build a response model straight from a SQLAlchemy object. Schemas for later phases are here already so you type this file once.

2.3 Routers

`app/routers/__init__.py` — empty.

`app/routers/deps.py`

```
import uuid

from fastapi import HTTPException
from sqlalchemy.orm import Session

from app.models import Client
```

```
def get_client_or_404(db: Session, client_id: uuid.UUID) -> Client:
    client = db.get(Client, client_id)
    if client is None:
        raise HTTPException(404, "Client not found")
    return client
```

`app/routers/auth.py`

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.db import get_db
from app.models import User
from app.schemas import LoginIn, TokenOut, UserOut
from app.security import create_token, current_user, verify_password

router = APIRouter(prefix="/auth", tags=["auth"])


@router.post("/login", response_model=TokenOut)
def login(body: LoginIn, db: Session = Depends(get_db)):
    user = db.scalar(select(User).where(User.email == body.email.lower()))
    if user is None or not verify_password(body.password, user.password_hash):
        raise HTTPException(401, "Bad email or password")
    return TokenOut(token=create_token(db, user), role=user.role, client_id=user.client_id)


@router.get("/me", response_model=UserOut)
def me(user: User = Depends(current_user)):
    return user
```

`app/routers/admin.py`

```
import uuid

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.config import settings
from app.db import get_db
from app.models import Client, User
from app.routers.deps import get_client_or_404
from app.schemas import ClientCreate, ClientOut, ClientUpdate, UserCreate, UserOut
from app.security import hash_password, require_admin

router = APIRouter(prefix="/admin", tags=["admin"], dependencies=[Depends(require_admin)])
```

```
@router.get("/clients", response_model=list[ClientOut])
def list_clients(db: Session = Depends(get_db)):
    return db.scalars(select(Client).order_by(Client.created_at)).all()


@router.post("/clients", response_model=ClientOut, status_code=201)
def create_client(body: ClientCreate, db: Session = Depends(get_db)):
    if db.scalar(select(Client).where(Client.slug == body.slug)):
        raise HTTPException(409, "Slug already used")
    client = Client(slug=body.slug, name=body.name, model=body.model or settings.default_model)
    db.add(client)
    db.commit()
    db.refresh(client)
    return client


@router.patch("/clients/{client_id}", response_model=ClientOut)
def update_client(client_id: uuid.UUID, body: ClientUpdate, db: Session = Depends(get_db)):
    client = get_client_or_404(db, client_id)
    if body.name is not None:
        client.name = body.name
    if body.model is not None:
        client.model = body.model
    db.commit()
    return client


@router.get("/users", response_model=list[UserOut])
def list_users(db: Session = Depends(get_db)):
    return db.scalars(select(User).order_by(User.created_at)).all()


@router.post("/users", response_model=UserOut, status_code=201)
def create_user(body: UserCreate, db: Session = Depends(get_db)):
    if body.role == "member" and body.client_id is None:
        raise HTTPException(422, "Members must belong to a client")
    if body.client_id is not None:
        get_client_or_404(db, body.client_id)
    email = body.email.lower()
    if db.scalar(select(User).where(User.email == email)):
        raise HTTPException(409, "Email already used")
    user = User(
        email=email,
        password_hash=hash_password(body.password),
        role=body.role,
        client_id=body.client_id,
```

```
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user
```

Explanation. `dependencies=[Depends(require_admin)]` on the router guards every route in the file. Changing a client's `model` here is all it takes to switch the LLM for that client: phase 5's registry includes the model in its cache key, so the next message recompiles the graph with the new model.

2.4 App entry point and admin seed

`app/main.py` (phase 5 replaces this with a version that starts the graph registry)

```
from fastapi import FastAPI

from app.routers import admin, auth

app = FastAPI(title="Support Agent")
app.include_router(auth.router)
app.include_router(admin.router)


@app.get("/health")
def health():
    return {"ok": True}
```

`scripts/seed_admin.py`

```
from sqlalchemy import select

from app.config import settings
from app.db import SessionLocal
from app.models import User
from app.security import hash_password

with SessionLocal() as db:
    email = settings.admin_email.lower()
    if db.scalar(select(User).where(User.email == email)):
        print("admin already exists:", email)
    else:
        db.add(User(email=email, password_hash=hash_password(settings.admin_password),
role="admin"))
        db.commit()
        print("admin created:", email)
```

Verify

Terminal 1:

```
uv run python -m scripts.seed_admin && uv run uvicorn app.main:app --reload --port 8000
```

Terminal 2 (replace the password with the one in your `.env`):

```
TOKEN=$(curl -s -X POST localhost:8000/auth/login -H 'content-type: application/json' -d '{"email":"admin@example.com","password":"change-me-now"}' | python3 -c 'import sys,json; print(json.load(sys.stdin)["token"])') && echo "$TOKEN"
```

```
curl -s -X POST localhost:8000/admin/clients -H "authorization: Bearer $TOKEN" -H 'content-type: application/json' -d '{"slug":"acme","name":"Acme Inc","model":"gpt-4.1-mini"}'
```

Copy the `id` from that output into the next command:

```
curl -s -X POST localhost:8000/admin/users -H "authorization: Bearer $TOKEN" -H 'content-type: application/json' -d '{"email":"sam@acme.com","password":"sam-password-1","role":"member","client_id":"PASTE-CLIENT-ID"}'
```

Expected: a client JSON with `slug: acme`, a user JSON with `role: member`, and `curl localhost:8000/admin/clients` without a token returns 401. Open http://localhost:8000/docs to see FastAPI's generated API explorer.

Commit: `git add -A && git commit -m "phase 2: auth and admin api"`

Phase 3 — The DAG spec

Goal: a strict, validated schema for the “DAG file”, a versioned API to save and activate DAGs per client, and unit tests for the validator.

What you learn: Pydantic model validators, graph theory you can type in 20 lines (Kahn’s algorithm for cycle detection), why validation at save time beats failing at chat time, versioning as append-only rows.

3.1 What a DAG file looks like

`examples/support_dag.json`

```
{
  "name": "acme-support-v1",
  "entry": "retrieve",
  "nodes": [
    {"id": "retrieve", "type": "retriever", "top_k": 4},
    {
      "id": "triage",
      "type": "router",
      "prompt": "You are the triage desk of a software company's support team. Classify the
customer's latest message.",
      "labels": ["billing", "technical", "general"]
    },
    {
      "id": "billing",
      "type": "agent",
      "system_prompt": "You are Acme's billing specialist. Answer only billing questions
(invoices, refunds, plans). Use the knowledge base excerpts and the search_knowledge tool before
answering. If you cannot answer, say exactly what you will escalate.",
      "tools": ["search_knowledge"]
    },
    {
      "id": "technical",
      "type": "agent",
      "system_prompt": "You are Acme's technical support engineer. Diagnose step by step, ask
one clarifying question at a time, and cite the knowledge base when you use it.",
      "tools": ["search_knowledge", "current_time"]
    },
    {
      "id": "general",
      "type": "agent",
      "system_prompt": "You are Acme's friendly front desk. Answer general questions briefly and
point the customer to billing or technical support when needed."
    }
  ],
  "edges": [
```

```
    {"source": "retrieve", "target": "triage"},
    {"source": "triage", "routes": {"billing": "billing", "technical": "technical", "general":
"general"}},
    {"source": "billing", "target": "END"},
    {"source": "technical", "target": "END"},
    {"source": "general", "target": "END"}
  ]
}
```

Three node types:

type	what it does	fields
`retriever`	pgvector search on the latest user message, puts hits in `state.context`	`top_k`
`router`	asks the LLM for exactly one label, puts it in `state.route`	`prompt`, `labels`
`agent`	an LLM persona with optional tools; appends its answer to `state.messages`	`system_prompt`, `tools`, `max_tool_rounds`

Every node may set `model` to override the client's model. Edges are either plain (`target`) or conditional (`routes`: label → node). `"END"` is the exit.

3.2 Schema + validator

`app/dag/__init__.py` — empty.

`app/dag/spec.py`

```
from typing import Literal

from pydantic import BaseModel, Field, model_validator

NodeType = Literal["agent", "router", "retriever"]
END = "END"


class NodeSpec(BaseModel):
    id: str = Field(pattern=r"^[a-z][a-z0-9_]*$")
    type: NodeType
    model: str | None = None            # override the client's model for this node only
    # agent
    system_prompt: str | None = None
    tools: list[str] = []
    max_tool_rounds: int = Field(default=3, ge=0, le=10)
    # router
    prompt: str | None = None
    labels: list[str] = []
```

```
    # retriever
    top_k: int = Field(default=4, ge=1, le=20)

    @model_validator(mode="after")
    def check_type_fields(self):
        if self.type == "agent" and not self.system_prompt:
            raise ValueError(f"agent node '{self.id}' needs a system_prompt")
        if self.type == "router":
            if not self.prompt:
                raise ValueError(f"router node '{self.id}' needs a prompt")
            if len(self.labels) < 2 or len(set(self.labels)) != len(self.labels):
                raise ValueError(f"router node '{self.id}' needs at least two distinct labels")
        return self


class EdgeSpec(BaseModel):
    source: str
    target: str | None = None               # plain edge
    routes: dict[str, str] | None = None    # conditional edge: label -> node id or "END"

    def targets(self) -> list[str]:
        return [self.target] if self.target is not None else list(self.routes.values())


class DagSpec(BaseModel):
    name: str
    entry: str
    nodes: list[NodeSpec]
    edges: list[EdgeSpec]

    @model_validator(mode="after")
    def check_graph(self):
        from app.dag.tools import TOOL_REGISTRY  # local import: tools.py imports the DB layer

        ids = [n.id for n in self.nodes]
        if len(ids) != len(set(ids)):
            raise ValueError("duplicate node ids")
        by_id = {n.id: n for n in self.nodes}
        if self.entry not in by_id:
            raise ValueError(f"entry '{self.entry}' is not a node")

        for n in self.nodes:
            for t in n.tools:
                if t not in TOOL_REGISTRY:
                    raise ValueError(f"node '{n.id}' uses unknown tool '{t}'")

        outgoing: dict[str, list[EdgeSpec]] = {i: [] for i in ids}
        for e in self.edges:
```

```
            if e.source not in by_id:
                raise ValueError(f"edge source '{e.source}' is not a node")
            if (e.target is None) == (e.routes is None):
                raise ValueError(f"edge from '{e.source}' needs exactly one of target / routes")
            for t in e.targets():
                if t != END and t not in by_id:
                    raise ValueError(f"edge from '{e.source}' points to unknown node '{t}'")
            outgoing[e.source].append(e)

        for n in self.nodes:
            outs = outgoing[n.id]
            if len(outs) != 1:
                raise ValueError(f"node '{n.id}' must have exactly one outgoing edge, has
{len(outs)}")
            e = outs[0]
            if n.type == "router":
                if e.routes is None:
                    raise ValueError(f"router '{n.id}' needs a routes edge")
                if set(e.routes) != set(n.labels):
                    raise ValueError(f"routes of '{n.id}' must cover exactly its labels
{n.labels}")
            elif e.routes is not None:
                raise ValueError(f"only router nodes may have routes ('{n.id}')")

        assert_acyclic(outgoing)
        return self

    def node(self, node_id: str) -> NodeSpec:
        return next(n for n in self.nodes if n.id == node_id)


def assert_acyclic(outgoing: dict[str, list[EdgeSpec]]) -> None:
    """Kahn's algorithm: repeatedly remove nodes with no incoming edges.
    If some nodes can never be removed, they sit on a cycle."""
    successors = {i: set() for i in outgoing}
    indegree = {i: 0 for i in outgoing}
    for src, edges in outgoing.items():
        for e in edges:
            for t in e.targets():
                if t != END and t not in successors[src]:
                    successors[src].add(t)
                    indegree[t] += 1
    ready = [i for i, d in indegree.items() if d == 0]
    removed = 0
    while ready:
        n = ready.pop()
        removed += 1
        for t in successors[n]:
```

```
            indegree[t] -= 1
            if indegree[t] == 0:
                ready.append(t)
    if removed != len(outgoing):
        stuck = sorted(i for i, d in indegree.items() if d > 0)
        raise ValueError(f"DAG contains a cycle through {stuck}")
```

Explanation.

- `model_validator(mode="after")` runs after field parsing, so you can reason about the whole object. Raising `ValueError` turns into a normal Pydantic validation error that FastAPI reports as 422.
- “Exactly one outgoing edge per node” is deliberately strict. It keeps the graph readable and means every node’s next step is unambiguous; branching only happens via a router’s `routes`.
- The cycle check is the heart of “DAG”. Kahn’s algorithm is also how you’d compute a topological order, which is handy if you later want to draw the graph.
- Tools are validated against a registry (phase 5 defines it) so a typo in the DAG file fails at save time, not when a customer is waiting.

Until phase 5 exists, create a stub so the import works:

`app/dag/tools.py` (temporary; phase 5 replaces it)

```
TOOL_REGISTRY: dict = {"search_knowledge": None, "current_time": None}
```

3.3 Versioned DAG API

`app/routers/dags.py`

```
import uuid
from typing import Any

from fastapi import APIRouter, Depends, HTTPException
from pydantic import ValidationError
from sqlalchemy import func, select, update
from sqlalchemy.orm import Session

from app.dag.spec import DagSpec
from app.db import get_db
from app.models import DagVersion
from app.routers.deps import get_client_or_404
from app.schemas import DagIn, DagOut
from app.security import require_admin

router = APIRouter(prefix="/admin", tags=["dags"], dependencies=[Depends(require_admin)])


def validation_messages(err: ValidationError) -> list[str]:
    return [e["msg"].removeprefix("Value error, ") for e in err.errors()]
```

```
@router.post("/dags/validate")
def validate_dag(spec: dict[str, Any]):
    try:
        DagSpec.model_validate(spec)
    except ValidationError as err:
        return {"ok": False, "errors": validation_messages(err)}
    return {"ok": True, "errors": []}


@router.get("/clients/{client_id}/dags", response_model=list[DagOut])
def list_dags(client_id: uuid.UUID, db: Session = Depends(get_db)):
    get_client_or_404(db, client_id)
    stmt = select(DagVersion).where(DagVersion.client_id ==
client_id).order_by(DagVersion.version.desc())
    return db.scalars(stmt).all()


@router.post("/clients/{client_id}/dags", response_model=DagOut, status_code=201)
def create_dag(client_id: uuid.UUID, body: DagIn, db: Session = Depends(get_db)):
    client = get_client_or_404(db, client_id)
    try:
        DagSpec.model_validate(body.spec)
    except ValidationError as err:
        raise HTTPException(422, detail=validation_messages(err))
    latest = db.scalar(select(func.max(DagVersion.version)).where(DagVersion.client_id ==
client.id)) or 0
    dag = DagVersion(client_id=client.id, version=latest + 1, spec=body.spec, is_active=False)
    db.add(dag)
    if body.activate:
        activate(db, dag)
    db.commit()
    db.refresh(dag)
    return dag


@router.post("/clients/{client_id}/dags/{version}/activate", response_model=DagOut)
def activate_dag(client_id: uuid.UUID, version: int, db: Session = Depends(get_db)):
    get_client_or_404(db, client_id)
    dag = db.scalar(select(DagVersion).where(DagVersion.client_id == client_id,
DagVersion.version == version))
    if dag is None:
        raise HTTPException(404, "Version not found")
    activate(db, dag)
    db.commit()
    return dag
```

```
def activate(db: Session, dag: DagVersion) -> None:
    db.execute(update(DagVersion).where(DagVersion.client_id ==
dag.client_id).values(is_active=False))
    dag.is_active = True
```

Register it in `app/main.py`:

```
from app.routers import admin, auth, dags
...
app.include_router(dags.router)
```

Explanation. Saving never overwrites: it inserts `version + 1` and flips the active flag. “Update the DAG” in the admin panel is therefore “save a new version”; rolling back is “activate an old one”. The registry (phase 5) keys its cache on the active version’s id, so activation is the hot-reload trigger.

3.4 Tests

`tests/__init__.py` — empty.

`tests/test_spec.py`

```
import copy
import json
from pathlib import Path

import pytest
from pydantic import ValidationError

from app.dag.spec import DagSpec

EXAMPLE = json.loads((Path(__file__).parent.parent / "examples" /
"support_dag.json").read_text())


def test_example_is_valid():
    spec = DagSpec.model_validate(EXAMPLE)
    assert spec.entry == "retrieve"
    assert spec.node("triage").labels == ["billing", "technical", "general"]


def test_cycle_is_rejected():
    bad = copy.deepcopy(EXAMPLE)
    bad["edges"][2] = {"source": "billing", "target": "retrieve"}  # billing -> retrieve ->
triage -> billing
    with pytest.raises(ValidationError, match="cycle"):
        DagSpec.model_validate(bad)
```

```
def test_router_must_cover_all_labels():
    bad = copy.deepcopy(EXAMPLE)
    del bad["edges"][1]["routes"]["general"]
    with pytest.raises(ValidationError, match="cover exactly"):
        DagSpec.model_validate(bad)


def test_node_needs_exactly_one_outgoing_edge():
    bad = copy.deepcopy(EXAMPLE)
    bad["edges"].append({"source": "general", "target": "billing"})
    with pytest.raises(ValidationError, match="exactly one outgoing edge"):
        DagSpec.model_validate(bad)


def test_unknown_tool_is_rejected():
    bad = copy.deepcopy(EXAMPLE)
    bad["nodes"][2]["tools"] = ["launch_rockets"]
    with pytest.raises(ValidationError, match="unknown tool"):
        DagSpec.model_validate(bad)
```

Verify

```
uv run pytest -q
```

Then with the server running (`uv run uvicorn app.main:app --reload --port 8000`) and `$TOKEN` from phase 2 (log in again if the shell is new), replace the client id:

```
curl -s -X POST localhost:8000/admin/clients/PASTE-CLIENT-ID/dags -H "authorization: Bearer $TOKEN" -H 'content-type: application/json' -d "{\"spec\": $(cat examples/support_dag.json), \"activate\": true}" | head -c 300
```

```
curl -s -X POST localhost:8000/admin/dags/validate -H "authorization: Bearer $TOKEN" -H 'content-type: application/json' -d '{"name":"x","entry":"a","nodes":[{"id":"a","type":"retriever"}],"edges":[{"source":"a","target":"a"}]}'
```

Expected: 5 tests pass; the first curl returns a version 1 with `is_active: true`; the second returns `{"ok": false, "errors": ["DAG contains a cycle through ['a']"]}`.

Commit: `git add -A && git commit -m "phase 3: dag spec, validation, versions"`

Phase 4 — Knowledge base (pgvector)

Goal: admins paste documents per client; the system chunks, embeds and stores them; a search function returns the most similar chunks for a query.

What you learn: what an embedding is in practice, chunking with overlap, cosine distance in SQL via pgvector, tenant-scoped retrieval.

4.1 OpenAI clients

`app/llm.py`

```
from functools import lru_cache

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

from app.config import settings


def chat_model(model: str | None = None) -> ChatOpenAI:
    return ChatOpenAI(model=model or settings.default_model, api_key=settings.openai_api_key)


@lru_cache(maxsize=1)
def embeddings() -> OpenAIEmbeddings:
    return OpenAIEmbeddings(model=settings.embedding_model, api_key=settings.openai_api_key)
```

Explanation. We don't set `temperature`: some newer OpenAI models reject any value other than their default, and the client's model is user-chosen. One embeddings client is cached because it holds an HTTP connection pool.

4.2 Chunk, embed, search

`app/knowledge.py`

```
import uuid

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.llm import embeddings
from app.models import Chunk, Document


def chunk_text(text: str, size: int = 800, overlap: int = 120) -> list[str]:
    """Fixed-size windows with overlap so a sentence cut at a boundary still appears whole in
```

```
one chunk."""
    text = text.strip()
    pieces: list[str] = []
    start = 0
    while start < len(text):
        end = min(start + size, len(text))
        piece = text[start:end].strip()
        if piece:
            pieces.append(piece)
        if end == len(text):
            break
        start = end - overlap
    return pieces


def add_document(db: Session, client_id: uuid.UUID, title: str, text: str) -> Document:
    pieces = chunk_text(text)
    vectors = embeddings().embed_documents(pieces) if pieces else []
    doc = Document(client_id=client_id, title=title)
    for position, (piece, vector) in enumerate(zip(pieces, vectors)):
        doc.chunks.append(Chunk(client_id=client_id, position=position, content=piece,
embedding=vector))
    db.add(doc)
    db.commit()
    db.refresh(doc)
    return doc


def search(db: Session, client_id: uuid.UUID, query: str, k: int = 4) -> list[Chunk]:
    query_vector = embeddings().embed_query(query)
    stmt = (
        select(Chunk)
        .where(Chunk.client_id == client_id)
        .order_by(Chunk.embedding.cosine_distance(query_vector))
        .limit(k)
    )
    return list(db.scalars(stmt))
```

Explanation. `embed_documents` sends all chunks in one request and returns one vector per chunk. `cosine_distance` renders as `embedding <=> $1` in SQL; ordering by it ascending puts the most similar chunks first. The `where client_id` clause is the tenant wall: Acme's bot can never see Globex's documents. 800 characters is a sane default for support docs; make it a per-client setting later if you want.

4.3 Knowledge API

`app/routers/knowledge.py`

```
import uuid

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy import select
from sqlalchemy.orm import Session

from app import knowledge
from app.db import get_db
from app.models import Document
from app.routers.deps import get_client_or_404
from app.schemas import DocumentIn, DocumentOut, SearchHit
from app.security import require_admin

router = APIRouter(prefix="/admin", tags=["knowledge"], dependencies=[Depends(require_admin)])


@router.get("/clients/{client_id}/documents", response_model=list[DocumentOut])
def list_documents(client_id: uuid.UUID, db: Session = Depends(get_db)):
    get_client_or_404(db, client_id)
    stmt = select(Document).where(Document.client_id ==
client_id).order_by(Document.created_at.desc())
    return db.scalars(stmt).all()


@router.post("/clients/{client_id}/documents", response_model=DocumentOut, status_code=201)
def add_document(client_id: uuid.UUID, body: DocumentIn, db: Session = Depends(get_db)):
    client = get_client_or_404(db, client_id)
    return knowledge.add_document(db, client.id, body.title, body.text)


@router.delete("/clients/{client_id}/documents/{document_id}", status_code=204)
def delete_document(client_id: uuid.UUID, document_id: uuid.UUID, db: Session =
Depends(get_db)):
    doc = db.get(Document, document_id)
    if doc is None or doc.client_id != client_id:
        raise HTTPException(404, "Document not found")
    db.delete(doc)
    db.commit()


@router.get("/clients/{client_id}/search", response_model=list[SearchHit])
def search_documents(
    client_id: uuid.UUID,
    q: str = Query(min_length=1),
    k: int = Query(default=4, ge=1, le=20),
    db: Session = Depends(get_db),
):
```

```
    get_client_or_404(db, client_id)
    return knowledge.search(db, client_id, q, k)
```

Register in `app/main.py`:

```
from app.routers import admin, auth, dags, knowledge
...
app.include_router(knowledge.router)
```

`examples/acme_refunds.md` — a document to test with:

```
Acme refund policy

Monthly plans can be cancelled at any time; the current month is not refunded.
Annual plans are refundable in full within 14 days of purchase, and pro-rated after that
until day 90. After day 90 annual plans are not refundable.
Refunds are issued to the original payment method within 5 business days.
To request a refund, email billing@acme.example with your invoice number.

Acme plans and prices

Starter: 19 USD per month, 1 user, 5 GB storage.
Team: 49 USD per month, up to 10 users, 100 GB storage, priority support.
Business: 199 USD per month, unlimited users, 1 TB storage, SSO, dedicated manager.
```

Verify

Server running, `$TOKEN` set, client id pasted:

```
curl -s -X POST localhost:8000/admin/clients/PASTE-CLIENT-ID/documents -H "authorization: Bearer
$TOKEN" -H 'content-type: application/json' -d "$(python3 -c 'import json;
print(json.dumps({"title": "Refund policy", "text":
open("examples/acme_refunds.md").read()}))')"
```

```
curl -s "localhost:8000/admin/clients/PASTE-CLIENT-ID/search?
q=can%20I%20get%20my%20money%20back%20on%20a%20yearly%20plan&k=2" -H "authorization: Bearer
$TOKEN"
```

Expected: a document JSON, then two hits whose `content` contains the refund paragraph. Check the rows exist:

```
docker compose exec db psql -U support -d support -c 'select position, left(content, 40) from
chunks'
```

Commit: `git add -A && git commit -m "phase 4: knowledge base with pgvector"`

Phase 5 — Graph compiler (the heart)

Goal: turn a validated `DagSpec` into a running LangGraph graph: three node builders, two tools, a registry that caches compiled graphs and recompiles when the active DAG or the client's model changes, Postgres-backed conversation memory, and a terminal chat to prove it all works.

What you learn: LangGraph `StateGraph`, state reducers (`add_messages`), conditional edges, closures as node factories, the raw tool-calling loop, structured output with a dynamic `Literal`, checkpointers and `thread_id`, hot reload by cache key.

5.1 State

`app/dag/state.py`

```
from typing import Annotated, TypedDict

from langchain_core.messages import AnyMessage
from langgraph.graph.message import add_messages


class ChatState(TypedDict, total=False):
    messages: Annotated[list[AnyMessage], add_messages]   # full conversation, persisted by the
checkpointer
    route: str                                             # label chosen by the last router
node
    context: list[str]                                     # chunks found by the last retriever
node
```

Explanation. A node returns a *partial* dict. For most keys the new value replaces the old one. `messages` is annotated with the `add_messages` reducer, so a node returning `{"messages": [ai]}` *appends* instead of overwriting. The checkpointer saves this whole state after every node, keyed by `thread_id`.

5.2 Tools (replace the phase 3 stub)

`app/dag/tools.py`

```
import uuid
from datetime import datetime, timezone

from langchain_core.tools import tool

from app import knowledge
from app.db import SessionLocal


def make_search_knowledge(client_id: uuid.UUID):
```

```
    @tool
    def search_knowledge(query: str) -> str:
        """Search this company's own knowledge base (policies, prices, product details).
        Use it whenever the answer depends on company-specific facts."""
        with SessionLocal() as db:
            hits = knowledge.search(db, client_id, query, k=4)
        if not hits:
            return "No relevant documents found."
        return "\n\n---\n\n".join(h.content for h in hits)

    return search_knowledge


def make_current_time(client_id: uuid.UUID):
    @tool
    def current_time() -> str:
        """Return the current date and time in UTC."""
        return datetime.now(timezone.utc).isoformat(timespec="seconds")

    return current_time


# name used in the DAG file -> factory(client_id) -> LangChain tool
TOOL_REGISTRY = {
    "search_knowledge": make_search_knowledge,
    "current_time": make_current_time,
}
```

Explanation. `@tool` turns a function into something the model can call; the docstring becomes the tool description the model reads, and the type hints become its JSON schema. Each factory closes over `client_id`, so the tool is already scoped to the right tenant and the model never sees or chooses a client id. Adding a tool is: write a factory, add one line to the registry, name it in a DAG file.

5.3 Node builders

`app/dag/nodes.py`

```
from typing import Literal

from langchain_core.messages import AIMessage, BaseMessage, HumanMessage, SystemMessage,
ToolMessage
from pydantic import Field, create_model

from app import knowledge
from app.dag.spec import NodeSpec
from app.dag.state import ChatState
from app.dag.tools import TOOL_REGISTRY
from app.db import SessionLocal
```

```
from app.llm import chat_model
from app.models import Client


def message_text(message: BaseMessage) -> str:
    """Content can be a plain string or a list of content blocks; return the text either way."""
    content = message.content
    if isinstance(content, str):
        return content
    return "".join(b.get("text", "") for b in content if isinstance(b, dict) and b.get("type")
== "text")


def last_human_text(messages: list[BaseMessage]) -> str:
    for m in reversed(messages):
        if isinstance(m, HumanMessage):
            return message_text(m)
    return ""


def plain_turns(messages: list[BaseMessage], limit: int) -> list[BaseMessage]:
    """Only human messages and final AI answers, no tool traffic. Safe to slice: a ToolMessage
    without its preceding tool call would make the OpenAI API reject the request."""
    kept = [
        m for m in messages
        if isinstance(m, HumanMessage) or (isinstance(m, AIMessage) and not m.tool_calls)
    ]
    return kept[-limit:]


def build_retriever_node(node: NodeSpec, client: Client):
    def retrieve(state: ChatState) -> dict:
        query = last_human_text(state["messages"])
        with SessionLocal() as db:
            hits = knowledge.search(db, client.id, query, k=node.top_k)
        return {"context": [h.content for h in hits]}

    return retrieve


def build_router_node(node: NodeSpec, client: Client):
    Decision = create_model(
        "Decision",
        label=(Literal[tuple(node.labels)], Field(description="The single best label")),
    )
    llm = chat_model(node.model or client.model).with_structured_output(Decision)
    instructions = f"{node.prompt}\n\nAnswer with exactly one label from: {',
'.join(node.labels)}."
```

```
    def route(state: ChatState) -> dict:
        decision = llm.invoke([SystemMessage(instructions), *plain_turns(state["messages"], 6)])
        return {"route": decision.label}

    return route


def build_agent_node(node: NodeSpec, client: Client):
    tools = [TOOL_REGISTRY[name](client.id) for name in node.tools]
    tools_by_name = {t.name: t for t in tools}
    base_llm = chat_model(node.model or client.model)
    llm = base_llm.bind_tools(tools) if tools else base_llm

    def system_text(state: ChatState) -> str:
        text = node.system_prompt
        context = state.get("context") or []
        if context:
            excerpts = "\n\n".join(f"[{i + 1}] {c}" for i, c in enumerate(context))
            text += f"\n\n# Knowledge base excerpts\n{excerpts}"
        return text

    def agent(state: ChatState) -> dict:
        system = SystemMessage(system_text(state))
        history = list(state["messages"])
        new: list[BaseMessage] = []
        for _ in range(node.max_tool_rounds + 1):
            ai = llm.invoke([system, *history, *new])
            new.append(ai)
            if not ai.tool_calls:
                break
            for call in ai.tool_calls:
                result = tools_by_name[call["name"]].invoke(call["args"])
                new.append(ToolMessage(content=str(result), tool_call_id=call["id"]))
        if isinstance(new[-1], ToolMessage):
            # ran out of rounds with tool results pending: force a final answer without tools
            new.append(base_llm.invoke([system, *history, *new]))
        return {"messages": new}

    return agent


def build_node(node: NodeSpec, client: Client):
    builders = {
        "retriever": build_retriever_node,
        "router": build_router_node,
        "agent": build_agent_node,
```

```
    }
    return builders[node.type](node, client)
```

Explanation.

- Each builder is a closure factory: it does the expensive setup once (model object, tools, the structured-output schema) and returns the function LangGraph will call per turn. That function receives the state and returns a partial update.
- **Router.** `create_model` builds a Pydantic class at runtime whose `label` field is `Literal["billing", "technical", "general"]`. `with_structured_output` makes the model fill that schema, so the answer can only be one of the labels. The conditional edge then maps that label to a node.
- **Agent.** This is the ReAct loop with no magic. Call the model; if it asked for tools, run each, append `ToolMessage`s, call again. `max_tool_rounds` bounds it. The tail check handles the edge case where the limit hits while tool results are pending: the OpenAI API requires every tool call to be answered, and we also want the customer to get a reply, so we make one last call with the tool-less model.
- The retriever's hits go into `state.context`; the agent's system prompt gets them appended as numbered excerpts. The agent can *also* call `search_knowledge` with a better query of its own. Two retrieval styles, both in the DAG file, both scoped by client.

5.4 Compiler

`app/dag/compiler.py`

```
from langgraph.graph import END, START, StateGraph

from app.dag.nodes import build_node
from app.dag.spec import END as SPEC_END
from app.dag.spec import DagSpec
from app.dag.state import ChatState
from app.models import Client


def compile_dag(spec: DagSpec, client: Client, checkpointer):
    graph = StateGraph(ChatState)

    for node in spec.nodes:
        graph.add_node(node.id, build_node(node, client))

    graph.add_edge(START, spec.entry)
    for edge in spec.edges:
        if edge.routes is not None:
            graph.add_conditional_edges(
                edge.source,
                pick_route,
                {label: target(t) for label, t in edge.routes.items()},
            )
        else:
            graph.add_edge(edge.source, target(edge.target))
```

```
    return graph.compile(checkpointer=checkpointer)


def target(name: str):
    return END if name == SPEC_END else name


def pick_route(state: ChatState) -> str:
    return state["route"]
```

Explanation. This is the whole translation: spec nodes → `add_node`, plain edges → `add_edge`, routes → `add_conditional_edges(source, function, mapping)`. The function returns whatever the router wrote into `state["route"]`; the mapping turns that label into the next node. Validation already guaranteed the mapping is complete, so the graph can't dead-end.

5.5 Registry (cache + hot reload + memory)

`app/dag/registry.py`

```
import uuid

from langgraph.checkpoint.postgres import PostgresSaver
from psycopg_pool import ConnectionPool
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.config import settings
from app.dag.compiler import compile_dag
from app.dag.spec import DagSpec
from app.models import Client, DagVersion


class GraphRegistry:
    def __init__(self):
        self._pool: ConnectionPool | None = None
        self.checkpointer: PostgresSaver | None = None
        self._cache: dict[uuid.UUID, tuple[tuple, object, DagSpec]] = {}

    def start(self) -> None:
        self._pool = ConnectionPool(
            conninfo=settings.psycopg_url,
            min_size=1,
            max_size=10,
            kwargs={"autocommit": True, "prepare_threshold": 0},
            open=True,
        )
        self.checkpointer = PostgresSaver(self._pool)
```

```
        self.checkpointer.setup()   # creates LangGraph's checkpoint tables if missing

    def stop(self) -> None:
        if self._pool is not None:
            self._pool.close()

    def get(self, db: Session, client: Client):
        """Return (compiled_graph, spec) for the client's active DAG, compiling only when
needed."""
        active = db.scalar(
            select(DagVersion).where(DagVersion.client_id == client.id,
DagVersion.is_active.is_(True))
        )
        if active is None:
            raise LookupError(f"client '{client.slug}' has no active DAG")

        key = (active.id, client.model)
        cached = self._cache.get(client.id)
        if cached is not None and cached[0] == key:
            return cached[1], cached[2]

        spec = DagSpec.model_validate(active.spec)
        graph = compile_dag(spec, client, self.checkpointer)
        self._cache[client.id] = (key, graph, spec)
        return graph, spec


registry = GraphRegistry()
```

Explanation. The cache key is `(active version id, client model)`. Save a new DAG version → new id → recompile on the next message. Activate an old version → its id → recompile. Change the client's model in the admin panel → recompile. Nothing else to invalidate, no restart. The checkpointer uses its own psycopg pool because LangGraph talks to Postgres directly, not through SQLAlchemy; `autocommit=True` is what it requires.

5.6 Wire it into the app

Replace `app/main.py`:

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

from app.dag.registry import registry
from app.routers import admin, auth, dags, knowledge


@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
    registry.start()
    yield
    registry.stop()


app = FastAPI(title="Support Agent", lifespan=lifespan)
for module in (auth, admin, dags, knowledge):
    app.include_router(module.router)


@app.get("/health")
def health():
    return {"ok": True}
```

5.7 Terminal chat

`scripts/chat_cli.py`

```
import sys
import uuid

from langchain_core.messages import HumanMessage
from sqlalchemy import select

from app.dag.nodes import message_text
from app.dag.registry import registry
from app.db import SessionLocal
from app.models import Client


def main(slug: str) -> None:
    registry.start()
    thread_id = str(uuid.uuid4())
    print(f"chatting with '{slug}'  (thread {thread_id})  Ctrl-C to quit")
    try:
        while True:
            text = input("\nyou> ").strip()
            if not text:
                continue
            with SessionLocal() as db:
                client = db.scalar(select(Client).where(Client.slug == slug))
                if client is None:
                    sys.exit(f"no client with slug '{slug}'")
                graph, _spec = registry.get(db, client)
            config = {"configurable": {"thread_id": thread_id}}
            for update in graph.stream({"messages": [HumanMessage(text)]}, config,
```

```
stream_mode="updates"):
                for node_name, delta in update.items():
                    route = (delta or {}).get("route")
                    print(f"  [{node_name}]" + (f" -> {route}" if route else ""))
                    for m in (delta or {}).get("messages", []):
                        if m.type == "ai" and not m.tool_calls:
                            print(f"\nbot> {message_text(m)}")
                        elif m.type == "tool":
                            print(f"  (tool result: {message_text(m)[:60]}...)")
    except (KeyboardInterrupt, EOFError):
        print("\nbye")
    finally:
        registry.stop()


if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "acme")
```

Explanation. `stream_mode="updates"` yields one dict per finished node: `{node_name: what_it_returned}`. That's why you see `[retrieve]`, `[triage] -> billing`, `[billing]` in order: you're watching the DAG execute. `registry.get` runs on every turn, so if you save a new DAG in another terminal the *next* message already uses it. Memory works because every call uses the same `thread_id`.

Verify

Make sure phase 3's DAG is active and phase 4's document is loaded, then:

```
uv run python -m scripts.chat_cli acme
```

Try, in order:

1. `hi, what plans do you have?` — expect `[triage] -> general` or `billing`, and prices from the document.
2. `I bought the annual plan 20 days ago, can I get a refund?` — expect `-> billing`, a pro-rated answer, probably a `(tool result ...)` line.
3. `and what about 100 days?` — expect it to remember the context from message 2 (memory via checkpointer).
4. `the app crashes when I upload a file` — expect `-> technical`.

Hot reload test: in a second terminal, save a new DAG version where the `general` node's `system_prompt` ends with "Always end with a joke." (POST the edited JSON as in phase 3). Send another message in the CLI: the next `general` answer should end with a joke, with no restart.

Confirm memory landed in Postgres:

```
docker compose exec db psql -U support -d support -c 'select thread_id, count(*) from
checkpoints group by thread_id'
```

Commit: `git add -A && git commit -m "phase 5: graph compiler, registry, terminal chat"`

Phase 6 — Chat API with streaming

Goal: members (and admins) create conversations and send messages over HTTP; the reply streams back token by token as Server-Sent Events, with node events so the UI can show which agent is answering.

What you learn: SSE over a POST with `StreamingResponse`, LangGraph's `stream_mode=["updates", "messages"]`, filtering token streams by node, keeping a conversation log separate from graph state.

6.1 Chat router

`app/routers/chat.py`

```
import json
import uuid

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import StreamingResponse
from langchain_core.messages import AIMessageChunk, HumanMessage
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.dag.nodes import message_text
from app.dag.registry import registry
from app.db import SessionLocal, get_db
from app.models import Client, Conversation, Message, User
from app.schemas import ConversationCreate, ConversationOut, MessageIn, MessageOut
from app.security import current_user

router = APIRouter(prefix="/chat", tags=["chat"])


def conversation_or_404(db: Session, conversation_id: uuid.UUID, user: User) -> Conversation:
    conv = db.get(Conversation, conversation_id)
    if conv is None or (user.role != "admin" and conv.user_id != user.id):
        raise HTTPException(404, "Conversation not found")
    return conv


def sse(payload: dict) -> str:
    return f"data: {json.dumps(payload)}\n\n"


@router.get("/conversations", response_model=list[ConversationOut])
def list_conversations(user: User = Depends(current_user), db: Session = Depends(get_db)):
    stmt = select(Conversation).where(Conversation.user_id ==
user.id).order_by(Conversation.created_at.desc())
    return db.scalars(stmt).all()
```

```
@router.post("/conversations", response_model=ConversationOut, status_code=201)
def create_conversation(
    body: ConversationCreate,
    user: User = Depends(current_user),
    db: Session = Depends(get_db),
):
    client_id = body.client_id if (user.role == "admin" and body.client_id) else user.client_id
    if client_id is None:
        raise HTTPException(422, "Admins must pass client_id; members are bound to their
client")
    if db.get(Client, client_id) is None:
        raise HTTPException(404, "Client not found")
    conv = Conversation(client_id=client_id, user_id=user.id, title="New conversation")
    db.add(conv)
    db.commit()
    db.refresh(conv)
    return conv


@router.get("/conversations/{conversation_id}/messages", response_model=list[MessageOut])
def list_messages(
    conversation_id: uuid.UUID,
    user: User = Depends(current_user),
    db: Session = Depends(get_db),
):
    conversation_or_404(db, conversation_id, user)
    stmt = select(Message).where(Message.conversation_id ==
conversation_id).order_by(Message.created_at)
    return db.scalars(stmt).all()


@router.post("/conversations/{conversation_id}/messages")
def send_message(
    conversation_id: uuid.UUID,
    body: MessageIn,
    user: User = Depends(current_user),
    db: Session = Depends(get_db),
):
    conv = conversation_or_404(db, conversation_id, user)
    client = db.get(Client, conv.client_id)
    try:
        graph, spec = registry.get(db, client)
    except LookupError as err:
        raise HTTPException(409, str(err))
    agent_nodes = {n.id for n in spec.nodes if n.type == "agent"}
```

```
    db.add(Message(conversation_id=conv.id, role="user", content=body.content))
    if conv.title == "New conversation":
        conv.title = body.content[:60]
    db.commit()

    config = {"configurable": {"thread_id": str(conv.id)}}

    def events():
        reply: list[str] = []
        try:
            stream = graph.stream(
                {"messages": [HumanMessage(body.content)]},
                config,
                stream_mode=["updates", "messages"],
            )
            for mode, data in stream:
                if mode == "updates":
                    for node_name, delta in data.items():
                        yield sse({"type": "node", "name": node_name, "route": (delta or
{}).get("route")})
                else:
                    chunk, meta = data
                    if isinstance(chunk, AIMessageChunk) and meta.get("langgraph_node") in
agent_nodes:
                        text = message_text(chunk)
                        if text:
                            reply.append(text)
                            yield sse({"type": "token", "text": text})
        except Exception as err:  # noqa: BLE001 - surface anything to the UI instead of a dead
stream
            yield sse({"type": "error", "message": str(err)})
            return

        with SessionLocal() as session:
            session.add(Message(conversation_id=conv.id, role="assistant",
content="".join(reply)))
            session.commit()
        yield sse({"type": "done"})

    return StreamingResponse(
        events(),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
```

Explanation.

- `stream_mode=["updates", "messages"]` with a list makes LangGraph yield `(mode, data)` pairs. `"updates"` gives node completions (used for the “which agent” badges). `"messages"` gives `(message_chunk, metadata)` for every LLM token emitted inside *any* node, including the router’s structured output. `metadata["langgraph_node"]` tells you where it came from, so we only forward tokens from agent nodes.
- The generator is a plain sync function; Starlette iterates it in a threadpool, so the LLM calls don’t block the event loop. The request-scoped `db` session may already be closed by then, which is why the assistant message is saved with a fresh `SessionLocal()`.
- The `thread_id` is the conversation id. LangGraph loads the previous state for that thread before running, so memory is per conversation and per client with no extra code.
- The `messages` table is the human-readable log: the UI reloads history from it, the admin can read it. LangGraph’s checkpoints are the machine memory.

6.2 Register the router

In `app/main.py`:

```
from app.routers import admin, auth, chat, dags, knowledge
...
for module in (auth, admin, dags, knowledge, chat):
    app.include_router(module.router)
```

Verify

Server running. Log in as the member from phase 2:

```
MTOKEN=$(curl -s -X POST localhost:8000/auth/login -H 'content-type: application/json' -d
'{"email":"sam@acme.com","password":"sam-password-1"}' | python3 -c 'import sys,json;
print(json.load(sys.stdin)["token"])') && echo "$MTOKEN"
```

```
CONV=$(curl -s -X POST localhost:8000/chat/conversations -H "authorization: Bearer $MTOKEN" -H
'content-type: application/json' -d '{}' | python3 -c 'import sys,json;
print(json.load(sys.stdin)["id"])') && echo "$CONV"
```

```
curl -N -X POST "localhost:8000/chat/conversations/$CONV/messages" -H "authorization: Bearer
$MTOKEN" -H 'content-type: application/json' -d '{"content":"how much is the Team plan?"}'
```

Expected: a `node` event for `retrieve`, a `node` event for `triage` with a `route`, then many `token` events, then a `node` event for the agent, then `done`. Then:

```
curl -s "localhost:8000/chat/conversations/$CONV/messages" -H "authorization: Bearer $MTOKEN"
```

shows the user message and the full assistant reply.

Commit: `git add -A && git commit -m "phase 6: chat api with sse streaming"`

Phase 7 — Frontend: admin panel + chat

Goal: two static pages served by FastAPI. `/admin` onboards people, creates clients, edits DAG files and knowledge. `/` is the chat. No build step.

What you learn: consuming SSE from a POST with `fetch` + `ReadableStream`, keeping a token in `localStorage`, how little JS a working UI needs.

7.1 Serve the pages

In `app/main.py` add:

```
from pathlib import Path

from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles

STATIC = Path(__file__).resolve().parent.parent / "static"
```

and after the `include_router` loop:

```
app.mount("/static", StaticFiles(directory=STATIC), name="static")


@app.get("/", include_in_schema=False)
def chat_page():
    return FileResponse(STATIC / "chat.html")


@app.get("/admin", include_in_schema=False)
def admin_page():
    return FileResponse(STATIC / "admin.html")
```

`GET /admin` (the page) does not clash with `/admin/clients` (the API): FastAPI matches full paths.

7.2 Shared helper

`static/common.js`

```
const TOKEN_KEY = "support_token";

function getToken() { return localStorage.getItem(TOKEN_KEY); }
function setToken(t) { if (t) localStorage.setItem(TOKEN_KEY, t); else
localStorage.removeItem(TOKEN_KEY); }

async function api(path, { method = "GET", body } = {}) {
```

```
  const headers = { "content-type": "application/json" };
  const token = getToken();
  if (token) headers.authorization = `Bearer ${token}`;
  const res = await fetch(path, { method, headers, body: body === undefined ? undefined :
JSON.stringify(body) });
  if (res.status === 401) { setToken(null); location.reload(); return; }
  if (res.status === 204) return null;
  const data = await res.json();
  if (!res.ok) throw new Error(typeof data.detail === "string" ? data.detail :
JSON.stringify(data.detail));
  return data;
}

async function login(email, password) {
  const data = await api("/auth/login", { method: "POST", body: { email, password } });
  setToken(data.token);
  return data;
}

function el(tag, attrs = {}, ...children) {
  const node = document.createElement(tag);
  for (const [k, v] of Object.entries(attrs)) {
    if (k === "onclick") node.onclick = v;
    else if (k === "class") node.className = v;
    else node.setAttribute(k, v);
  }
  for (const c of children) node.append(c);
  return node;
}

const css = `
  body { font: 14px/1.4 system-ui, sans-serif; margin: 0; background: #f6f7f9; color: #1d2330; }
  header { background: #1d2330; color: #fff; padding: 10px 16px; display: flex; gap: 12px;
align-items: center; }
  header a { color: #cfd6e4; }
  button { padding: 6px 10px; border: 1px solid #c3cad6; background: #fff; border-radius: 6px;
cursor: pointer; }
  button.primary { background: #2f6df6; color: #fff; border-color: #2f6df6; }
  input, textarea, select { padding: 6px 8px; border: 1px solid #c3cad6; border-radius: 6px;
font: inherit; width: 100%; box-sizing: border-box; }
  textarea { min-height: 220px; font-family: ui-monospace, monospace; }
  .card { background: #fff; border: 1px solid #e1e5ec; border-radius: 8px; padding: 12px;
margin-bottom: 12px; }
  .row { display: flex; gap: 8px; margin-bottom: 8px; }
  .row > * { flex: 1; }
  .list button { display: block; width: 100%; text-align: left; margin-bottom: 4px; }
  .list button.active { border-color: #2f6df6; background: #e9f0ff; }
  .error { color: #b00020; white-space: pre-wrap; }
```

```
  .badge { display: inline-block; background: #e9f0ff; color: #2f6df6; border-radius: 10px;
padding: 1px 8px; font-size: 12px; margin-right: 4px; }
`;
document.head.append(el("style", {}, css));
```

7.3 Admin panel

`static/admin.html`

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Support Agent — Admin</title>
<script src="/static/common.js"></script>
<style>
  main { display: grid; grid-template-columns: 260px 1fr; gap: 16px; padding: 16px; }
  nav button { margin-right: 4px; }
</style>
</head>
<body>
<header><strong>Support Agent · Admin</strong><a href="/">chat</a><span style="flex:1"></span>
<button id="logout">Log out</button></header>
<div id="app"></div>
<script>
const app = document.getElementById("app");
let clients = [], users = [], selected = null, tab = "dag";

async function boot() {
  if (!getToken()) return renderLogin();
  const me = await api("/auth/me");
  if (me.role !== "admin") { app.replaceChildren(el("p", { class: "error" }, "Admins only. Log
out and use an admin account.")); return; }
  await reload();
}

async function reload() {
  [clients, users] = await Promise.all([api("/admin/clients"), api("/admin/users")]);
  if (selected) selected = clients.find(c => c.id === selected.id) || null;
  render();
}

function renderLogin() {
  const email = el("input", { placeholder: "email" }), pw = el("input", { placeholder:
"password", type: "password" });
  const err = el("div", { class: "error" });
```

```
  app.replaceChildren(el("div", { class: "card", style: "max-width:360px;margin:60px auto" },
    el("h3", {}, "Admin login"), el("div", { class: "row" }, email), el("div", { class: "row" },
pw), err,
    el("button", { class: "primary", onclick: async () => {
      try { await login(email.value, pw.value); boot(); } catch (e) { err.textContent =
e.message; }
    } }, "Log in")));
}

function render() {
  app.replaceChildren(el("main", {}, renderSidebar(), selected ? renderClient() : el("p", {},
"Select or create a client.")));
}

function renderSidebar() {
  const slug = el("input", { placeholder: "slug (e.g. acme)" }), name = el("input", {
placeholder: "Client name" }), model = el("input", { placeholder: "model (blank = default)" });
  const err = el("div", { class: "error" });
  return el("div", {},
    el("div", { class: "card list" }, el("h3", {}, "Clients"),
      ...clients.map(c => el("button", { class: selected?.id === c.id ? "active" : "", onclick:
() => { selected = c; render(); } }, `${c.name} (${c.slug})`)),
    el("div", { class: "card" }, el("h4", {}, "New client"),
      el("div", { class: "row" }, slug), el("div", { class: "row" }, name), el("div", { class:
"row" }, model), err,
      el("button", { class: "primary", onclick: async () => {
        try { selected = await api("/admin/clients", { method: "POST", body: { slug: slug.value,
name: name.value, model: model.value || null } }); await reload(); }
        catch (e) { err.textContent = e.message; }
      } }, "Create")));
}

function renderClient() {
  const tabs = { settings: renderSettings, users: renderUsers, dag: renderDag, knowledge:
renderKnowledge };
  const nav = el("nav", { class: "card" }, ...Object.keys(tabs).map(t => el("button", { class: t
=== tab ? "primary" : "", onclick: () => { tab = t; render(); } }, t)));
  const body = el("div", { class: "card" });
  tabs[tab](body);
  return el("div", {}, el("h2", {}, `${selected.name} `, el("small", {}, `slug: ${selected.slug}
· model: ${selected.model}`)), nav, body);
}

function renderSettings(box) {
  const name = el("input", { value: selected.name }), model = el("input", { value:
selected.model }), err = el("div", { class: "error" });
  box.append(el("h3", {}, "Settings"), el("div", { class: "row" }, el("label", {}, "Name",
name), el("label", {}, "OpenAI model", model)), err,
```

```
    el("button", { class: "primary", onclick: async () => {
      try { await api(`/admin/clients/${selected.id}`, { method: "PATCH", body: { name:
name.value, model: model.value } }); await reload(); }
      catch (e) { err.textContent = e.message; }
    } }, "Save"));
}

function renderUsers(box) {
  const mine = users.filter(u => u.client_id === selected.id);
  const email = el("input", { placeholder: "email" }), pw = el("input", { placeholder: "password
(8+ chars)", type: "password" }), err = el("div", { class: "error" });
  box.append(el("h3", {}, "People"),
    el("ul", {}, ...mine.map(u => el("li", {}, `${u.email} · ${u.role}`))),
    el("h4", {}, "Onboard a member"), el("div", { class: "row" }, email, pw), err,
    el("button", { class: "primary", onclick: async () => {
      try { await api("/admin/users", { method: "POST", body: { email: email.value, password:
pw.value, role: "member", client_id: selected.id } }); await reload(); }
      catch (e) { err.textContent = e.message; }
    } }, "Create member"));
}

async function renderDag(box) {
  const versions = await api(`/admin/clients/${selected.id}/dags`);
  const active = versions.find(v => v.is_active);
  const editor = el("textarea", {}, active ? JSON.stringify(active.spec, null, 2) : "");
  const out = el("div", { class: "error" });
  const parse = () => { try { return JSON.parse(editor.value); } catch (e) { out.textContent =
"Not valid JSON: " + e.message; return null; } };
  box.append(el("h3", {}, "DAG file ", active ? el("span", { class: "badge" }, `active:
v${active.version}`) : el("span", { class: "badge" }, "no active DAG")),
    editor, out,
    el("div", { class: "row", style: "margin-top:8px" },
      el("button", { onclick: async () => {
        const spec = parse(); if (!spec) return;
        const r = await api("/admin/dags/validate", { method: "POST", body: spec });
        out.textContent = r.ok ? "Valid ✓" : r.errors.join("\n");
      } }, "Validate"),
      el("button", { class: "primary", onclick: async () => {
        const spec = parse(); if (!spec) return;
        try { await api(`/admin/clients/${selected.id}/dags`, { method: "POST", body: { spec,
activate: true } }); render(); }
        catch (e) { out.textContent = e.message; }
      } }, "Save as new version & activate")),
    el("h4", {}, "Versions"),
    el("ul", {}, ...versions.map(v => el("li", {}, `v${v.version} · ${v.spec.name} · ${new
Date(v.created_at).toLocaleString()} `,
      v.is_active ? el("span", { class: "badge" }, "active") : el("button", { onclick: async ()
=> { await api(`/admin/clients/${selected.id}/dags/${v.version}/activate`, { method: "POST" });
```

```
render(); } }, "activate"))));
}

async function renderKnowledge(box) {
  const docs = await api(`/admin/clients/${selected.id}/documents`);
  const title = el("input", { placeholder: "Title" }), text = el("textarea", { placeholder:
"Paste the document text" }), err = el("div", { class: "error" });
  const q = el("input", { placeholder: "Test a search query" }), hits = el("div", {});
  box.append(el("h3", {}, "Knowledge base"),
    el("ul", {}, ...docs.map(d => el("li", {}, `${d.title} `, el("button", { onclick: async ()
=> { await api(`/admin/clients/${selected.id}/documents/${d.id}`, { method: "DELETE" });
render(); } }, "delete")))),
    el("h4", {}, "Add document"), el("div", { class: "row" }, title), text, err,
    el("button", { class: "primary", onclick: async () => {
      try { await api(`/admin/clients/${selected.id}/documents`, { method: "POST", body: {
title: title.value, text: text.value } }); render(); }
      catch (e) { err.textContent = e.message; }
    } }, "Embed & save"),
    el("h4", {}, "Test search"), el("div", { class: "row" }, q, el("button", { onclick: async ()
=> {
      const r = await api(`/admin/clients/${selected.id}/search?
q=${encodeURIComponent(q.value)}&k=3`);
      hits.replaceChildren(...r.map(h => el("pre", { class: "card" }, h.content)));
    } }, "Search")), hits);
}

document.getElementById("logout").onclick = () => { setToken(null); location.reload(); };
boot();
</script>
</body>
</html>
```

7.4 Chat page

`static/chat.html`

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Support Agent — Chat</title>
<script src="/static/common.js"></script>
<style>
  main { display: grid; grid-template-columns: 240px 1fr; gap: 16px; padding: 16px; height:
calc(100vh - 44px); box-sizing: border-box; }
  #thread { display: flex; flex-direction: column; }
```

```
  #messages { flex: 1; overflow-y: auto; }
  .msg { max-width: 72%; padding: 8px 12px; border-radius: 12px; margin: 6px 0; white-space:
pre-wrap; }
  .user { background: #2f6df6; color: #fff; margin-left: auto; }
  .assistant { background: #fff; border: 1px solid #e1e5ec; }
  .trace { font-size: 12px; color: #667; margin: 2px 0 6px; }
</style>
</head>
<body>
<header><strong>Support Agent</strong><a href="/admin">admin</a><span style="flex:1"></span>
<button id="logout">Log out</button></header>
<div id="app"></div>
<script>
const app = document.getElementById("app");
let me = null, conversations = [], current = null, clientsForAdmin = [];

async function boot() {
  if (!getToken()) return renderLogin();
  me = await api("/auth/me");
  if (me.role === "admin") clientsForAdmin = await api("/admin/clients");
  conversations = await api("/chat/conversations");
  render();
}

function renderLogin() {
  const email = el("input", { placeholder: "email" }), pw = el("input", { placeholder:
"password", type: "password" }), err = el("div", { class: "error" });
  app.replaceChildren(el("div", { class: "card", style: "max-width:360px;margin:60px auto" },
    el("h3", {}, "Sign in"), el("div", { class: "row" }, email), el("div", { class: "row" },
pw), err,
    el("button", { class: "primary", onclick: async () => { try { await login(email.value,
pw.value); boot(); } catch (e) { err.textContent = e.message; } } }, "Sign in")));
}

function render() {
  const clientPick = me.role === "admin"
    ? el("select", {}, ...clientsForAdmin.map(c => el("option", { value: c.id }, `${c.name}`)))
    : null;
  const side = el("div", { class: "card list" },
    el("h3", {}, "Conversations"),
    ...(clientPick ? [el("div", { class: "row" }, clientPick)] : []),
    el("button", { class: "primary", onclick: async () => {
      current = await api("/chat/conversations", { method: "POST", body: { client_id: clientPick
? clientPick.value : null } });
      conversations = await api("/chat/conversations"); render(); } }, "+ New conversation"),
    ...conversations.map(c => el("button", { class: current?.id === c.id ? "active" : "",
onclick: () => { current = c; render(); } }, c.title)));
  const thread = el("div", { id: "thread", class: "card" });
```

```
  app.replaceChildren(el("main", {}, side, thread));
  if (current) renderThread(thread);
  else thread.append(el("p", {}, "Start a new conversation."));
}

async function renderThread(thread) {
  const messages = el("div", { id: "messages" });
  const input = el("input", { placeholder: "Type your message and press Enter" });
  const history = await api(`/chat/conversations/${current.id}/messages`);
  for (const m of history) messages.append(bubble(m.role, m.content));
  thread.append(messages, el("div", { class: "row", style: "margin-top:8px" }, input));
  messages.scrollTop = messages.scrollHeight;
  input.focus();
  input.onkeydown = async (ev) => {
    if (ev.key !== "Enter" || !input.value.trim()) return;
    const text = input.value.trim(); input.value = ""; input.disabled = true;
    messages.append(bubble("user", text));
    const trace = el("div", { class: "trace" });
    const reply = bubble("assistant", "");
    messages.append(trace, reply);
    await streamReply(text, trace, reply, messages);
    input.disabled = false; input.focus();
    if (current.title === "New conversation") { conversations = await
api("/chat/conversations"); current = conversations.find(c => c.id === current.id); render(); }
  };
}

function bubble(role, text) { return el("div", { class: `msg ${role}` }, text); }

async function streamReply(text, trace, reply, messages) {
  const res = await fetch(`/chat/conversations/${current.id}/messages`, {
    method: "POST",
    headers: { "content-type": "application/json", authorization: `Bearer ${getToken()}` },
    body: JSON.stringify({ content: text }),
  });
  if (!res.ok) { reply.textContent = `Error ${res.status}: ${await res.text()}`; return; }
  const reader = res.body.getReader(), decoder = new TextDecoder();
  let buffer = "";
  while (true) {
    const { value, done } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });
    let idx;
    while ((idx = buffer.indexOf("\n\n")) >= 0) {
      const frame = buffer.slice(0, idx); buffer = buffer.slice(idx + 2);
      for (const line of frame.split("\n")) {
        if (!line.startsWith("data: ")) continue;
        const ev = JSON.parse(line.slice(6));
```

```
        if (ev.type === "node") trace.append(el("span", { class: "badge" }, ev.route ?
`${ev.name} → ${ev.route}` : ev.name));
        else if (ev.type === "token") reply.textContent += ev.text;
        else if (ev.type === "error") reply.textContent += `\n[error] ${ev.message}`;
        messages.scrollTop = messages.scrollHeight;
      }
    }
  }
}

document.getElementById("logout").onclick = () => { setToken(null); location.reload(); };
boot();
</script>
</body>
</html>
```

Explanation.

- The SSE stream comes from a POST, so the browser's `EventSource` (GET only) can't be used. Instead we read the response body as a stream and split it on the blank line that ends each SSE frame. The decoder is called with `{stream: true}` so a multi-byte character split across two network chunks is not corrupted.
- The trace line above each answer shows the DAG executing: `retrieve`, `triage → billing`, `billing`. That is your debugging view for routing problems.
- Admins pick a client from a dropdown and can test any client's bot; members are bound to their client by the API, not by the UI.

Verify

```
uv run uvicorn app.main:app --reload --port 8000
```

1. Open http://localhost:8000/admin, log in as admin. Create a second client (e.g. `globex`), paste the example DAG with different system prompts, add a document, onboard a member.
2. Open http://localhost:8000/ in a private window, log in as that member, start a conversation, ask something from the document. Watch the trace badges and the streamed answer.
3. Back in the admin panel, edit the DAG (change a system prompt), save. Send another message in the chat: the behaviour changes without a restart.
4. Change the client's model in Settings, send another message: still works, now on the new model.

Commit: `git add -A && git commit -m "phase 7: admin panel and chat ui"`

You now have the thing you set out to build. Phase 8 is the backlog.

Phase 8 — After it works

Only start these once phases 0–7 are green. Each is small and independent.

8.1 See inside the graph: LangSmith tracing (15 minutes)

Add to `.env`:

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_...
LANGSMITH_PROJECT=support-agent
```

and `os.environ` must actually contain them, so in `app/config.py` add fields `langsmith_tracing`, `langsmith_api_key`, `langsmith_project` (all optional) and at the bottom of `app/llm.py` export them with `os.environ.setdefault(...)` when set. Every chat turn then appears as a tree: router call, tool calls, final answer, with latency and token counts.

8.2 Schema migrations: Alembic

`create_all` won't add a column to an existing table. When you first need to change a model: `uv add alembic`, `uv run alembic init migrations`, point `env.py` at `app.db.Base.metadata` and `settings.database_url`, then `uv run alembic revision --autogenerate -m "..."` and `uv run alembic upgrade head`.

8.3 Long conversations

State grows forever. In `build_agent_node`, replace `history = list(state["messages"])` with `langchain_core.messages.trim_messages(..., max_tokens=..., token_counter=llm, start_on="human", include_system=False)`. Trim at the model-call boundary, not in the state, so the log stays complete.

8.4 New node types

Each one is: a field or two in `NodeSpec`, a builder in `nodes.py`, one line in `build_node`. - `handoff`: appends a fixed AI message ("A human will take over") and writes a `handoff` row. Routers can route to it. - `summarizer`: writes a one-line summary of the conversation into `conversations.title`. - `guard`: a cheap model that checks the incoming message for prompt-injection or abuse and routes to a refusal.

8.5 Per-client knowledge upload

Accept `.md` / `.txt` file uploads (`python-multipart`) instead of pasting; later PDFs via `pypdf`.

8.6 Evals

`tests/evals/cases.jsonl` with `{"message": ..., "expect_route": ...}` lines and a pytest that runs only the router node (cheap) and asserts the label. Run it whenever you change a triage prompt. Routing accuracy is the single most important number in this system.

8.7 Ship it

- Dockerfile for the API (`uv sync --frozen`, `uvicorn app.main:app --host 0.0.0.0`), add it to `docker-compose.yml` next to `db`.
- Put the real admin password and OpenAI key in the host's env, never in the image.
- Add `ALLOWED_ORIGINS` and CORS middleware only if the frontend moves to its own domain.